Lecture 7
Solutions

Homework 7: Review and merge

Solutions to Homework 7

Part A.

```
$ bash codeHints/L07/setup_hw7.sh
$ cd practice
$ git clone hw7_server.git hw7
$ cd hw7
$ git fetch
$ git log --oneline main..origin/teammate-feature     # 2 commits
$ git diff main...origin/teammate-feature             # "Helo, world" in greeting.txt
$ printf 'commits=2\ntypo-in=greeting.txt\nmerge-button=merge\n' > ../hw7_answers.txt
$ git switch teammate-feature
$ sed -i.bak 's/Helo/Hello/' greeting.txt && rm greeting.txt.bak    # or edit by hand
$ git commit -am "Fix typo in greeting"
$ git push
$ git switch main
$ git merge --no-ff --no-edit teammate-feature
$ git push
$ git branch -d teammate-feature
$ git push origin --delete teammate-feature
```

7.2. The branch adds two commits (`Add greeting` and `Document greeting`). The typo is `Helo` in `greeting.txt`. (`git diff` with three dots shows only what the branch added.)

7.3. `git switch teammate-feature` finds no local branch of that name, but there is `origin/teammate-feature`, so Git creates a tracking branch for you. A plain `git push` then updates the remote branch, as when you push a fix to an open pull request.

7.4. `--no-ff` forces a merge commit even though a fast-forward would be possible. This is what GitHub's button *Create a merge commit* does, hence the answer `merge`. (The checker expects a merge commit at the tip of the server's `main`; a fast-forward would fail.)

7.5. `git branch -d` works because the branch is merged. `git push origin --delete` removes it on the server (on GitHub, the *Delete branch* button).

Part B. Replace `A`, `B` by your account names.

```
Round 1 (collaborator)                              who
------------------------------------------------------------
create public repo pr-practice with README          A
Settings > Collaborators > Add people: B            A
accept the invitation                               B
git clone git@github.com:A/pr-practice.git practice/hw7_pr   B
git switch -c add-b-note                            B
(create notes-from-b.md) git add / git commit       B
git push -u origin add-b-note                       B
open the pull request (title + description)         B
inline comment, "Request changes"                  A
(edit file) git commit; git push                    B
"Approve", "Create a merge commit"                  A
"Delete branch"                                      B
```

```
git switch main; git pull; git branch -d add-b-note   B
printf 'my-pr=A/pr-practice#1\n' > practice/hw7_pr_answers.txt   B

Round 2 (fork)
------------------------------------------------------------
create public repo pr-practice-b with README    B
Fork                                             A
git clone git@github.com:A/pr-practice-b.git practice/hw7_pr   A
git remote add upstream git@github.com:B/pr-practice-b.git     A
git switch -c add-a-note; add file; commit      A
git push -u origin add-a-note                   A
open the PR (base: B's repo, head: A's fork)    A
review with a suggestion, "Approve",
"Squash and merge"                               B
git fetch upstream; git switch main             A
git merge upstream/main; git push origin main   A
printf 'my-pr=B/pr-practice-b#1\n' > practice/hw7_pr_answers.txt   A
```

Each of you writes the pull request *you opened*: **B** the one from Round 1 (in A's repository), **A** the one from Round 2 (in B's repository). Both repositories must be public, because the checker asks GitHub's public service whether the pull request is merged. The `upstream` check applies only to the fork route (A). After Round 2, A's `practice/hw7_pr` is the fork clone, so `origin` points at A's fork on github.com and `upstream` at B's repository.

Common mistakes.

- Merging with a fast-forward in 7.4 (`git merge teammate-feature` without `--no-ff`): the server's `main` then has no merge commit and the checker fails 7.4. Reset and redo.
- Opening the pull request from the fork *to the fork* (the default base on a fork can be your own repository). Check that the base repository is the original owner's.
- Forgetting `upstream`, then wondering why `git fetch upstream` fails ("not a git repository" or "no such remote").
- Writing the pull request number of your *partner's* request, or a private repository, in `hw7_pr_answers.txt`.
- `git branch -d` refusing after *Squash and merge*: expected, see the lecture; if the pull request really is merged, `git branch -D` is fine.